Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Daniel Saavedra Anderson Coello Maria Valarezo Jimmy Ontaneda
Asignatura	Simulación
Ciclo	5
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	004
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	José O. Guamán Q.
Fecha	22 de Octubre de 2026 23 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación.

- Implementar un modelo sencillo en diferentes herramientas.
- Comparar sintaxis, facilidad de implementación y visualización.
- Identificar el ámbito de aplicación de cada herramienta

3. Materiales y reactivos:

- Computador.
- Python.
- NetLogo.
- Java/JDK.

4. Equipos y Herramientas

- SimPy.

- NumPy.
- Matplotlib.
- NetLogo.
- JDK.
- Sistema operativo Linux o macOS

5. Procedimiento / Metodología Ejecutada

5.1 Arquitectura del Software (Patrón MVC)

```
src/
└── sir/
    ├── Main.java              Punto de entrada (arranque/argumentos)
    ├── model/                 MODELO: datos y logica de la simulacion
    │   ├── Estado.java        Estados S, I, R (enum)
    │   ├── Individuo.java     Individuo de la poblacion (id + estado)
    │   ├── ParametrosSIR.java N, I0, beta, gamma, tiempo, R0
    │   ├── ResultadoPaso.java Registro (tiempo, S, I, R)
    │   └── SimulacionSIR.java Poblacion + algoritmo por pasos
    ├── controller/            CONTROLADOR: orquesta la simulacion
    │   └── ControladorSIR.java Ejecuta los 100 pasos, exporta CSV y grafica
    └── view/
        ├── VistaConsola.java  Tabla de resultados en consola
        └── GraficaSIR.java    Grafica S(t), I(t), R(t) (PNG + ventana)
```

5.2 Pasos Metodológicos Ejecutados

1. Definición del problema

Se evaluará un modelo epidemiológico SIR por medio de una simulación hecha en Java, con el objetivo de visualizar una enfermedad infecciosa a lo largo del tiempo y como esta se va propagando.

2. Formulacion conceptual

El modelo matemático SIR aplicado en un escenario real, tiene varios puntos en los que se divide la población a estudiar:

- S: individuos sanos que no tiene la enfermedad, pero pueden contagiarse
- I: individuos enfermos que tienen la capacidad de transmitir el virus a los individuos sanos.
- R: individuos que superaron la enfermedad.

El total de la población se determina por: $N = S + I + R$. Además, se toman en cuenta las siguientes variables:

- β (tasa de transmisión): Mide la probabilidad de que ocurra un contagio cuando un individuo sano entra en contacto con un infectado
- γ (tasa de recuperación): Mide la proporción de infectados que se recuperan por unidad de tiempo.
- Tiempo: se define la propagación de la enfermedad a lo largo del tiempo.

3. Recolección y análisis de datos

Los datos usados para la simulación son:

Población: 100

Infectados iniciales: 5

Susceptibles iniciales: 95

β (tasa de transmisión): 0.30

γ (tasa de recuperación): 0.10

Tiempo de simulación: 100

4. Construcción del modelo

Se usó el lenguaje Java para realizar la simulación, siguiendo un patrón arquitectónico MVC, además, de usar la herramienta de visualización de datos NetLogo

5. Verificación y validación

Se comprobaron límites de parámetros, reproducción de semillas, conservación de N, continuidad temporal, transiciones, igualdad entre detalle y agregado, protección de archivos, rutas y comportamiento del visor.

Grupo de pruebas	Casos distintos	Fallos	Errores	Omitidos
Modelo, parámetros, opciones y archivos	187	0	0	0
JAR y terminal en procesos separados	7	0	0	0
Motor NetLogo sin pantalla	72	0	0	0
Ventana Swing y componente NetLogo	34	0	0	0
Total	**300**	**0**	**0**	**0**

Se ejecutó cada escenario una vez, con sus parámetros registrados en archivos .properties. Se extrajeron conteos finales, máximo de infectados, paso del pico y primer paso sin infectados. Se compararon las curvas y se reprodujeron estados en NetLogo.

La comparación de plataformas exige controlar reglas temporales, activación y aleatoriedad, además de parámetros[1]. En esta aplicación NetLogo no calcula otra trayectoria: representa la salida Java. Por ello, los controles del visor se revisaron como mecanismos de reproducción y no como un segundo experimento epidemiológico.

6. Resultados esperados

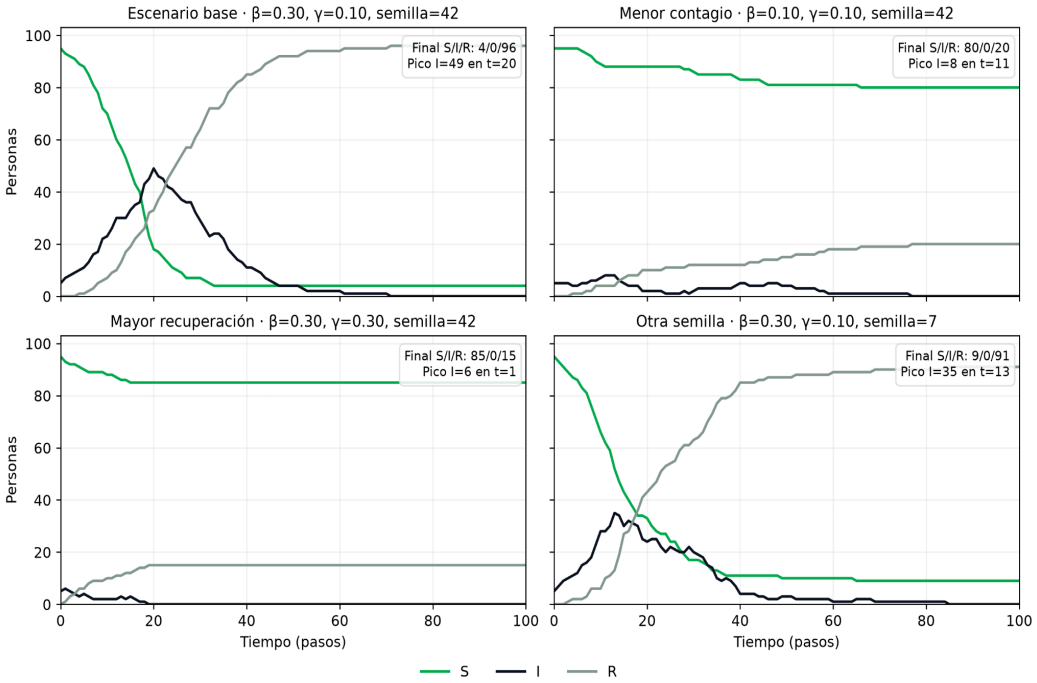
6.1. Evolución de los cuatro escenarios

En todos los escenarios se utilizaron N=100, S_0=95, I_0=5, R_0=0, un contacto por infectado y 100 pasos.

Escenario	β	γ	Semilla	S/I/R final	Pico I	Paso(s) del pico	Primer t con I=0
Escenario base	0.30	0.10	42	4 / 0 / 96	49	20	71

Menor contagio	0.10	0.10	42	80 / 0 / 20	8	11–13	77
Mayor recuperación	0.30	0.30	42	85 / 0 / 15	6	1	19
Otra semilla	0.30	0.10	7	9 / 0 / 91	35	13	85

Los valores finales corresponden a t=100. En menor contagio el máximo de 8 infectados se mantiene durante los pasos 11, 12 y 13. El primer t con I=0 indica cuándo deja de haber infectados en esa trayectoria; la simulación sigue registrando los estados hasta completar el horizonte.



En la base, 96 personas pasaron por la infección, incluidas las cinco iniciales: hubo 91 contagios nuevos. Al reducir β a 0.10, esa trayectoria terminó con 20 recuperados y 80 susceptibles; al aumentar γ a 0.30, terminó con 15 recuperados y 85 susceptibles. Los picos observados fueron menores que en la base. La infección se extinguió antes o después según el escenario: el menor pico no garantiza por sí solo una extinción más temprana.

Cambiar únicamente la semilla a 7 produjo un pico de 35 en t=13 y 91 recuperados finales. Esto muestra variabilidad entre realizaciones del mismo modelo. Los resultados describen cuatro trayectorias: no se calcularon promedios entre réplicas, intervalos de confianza ni significación estadística.

6.2 Tabla comparativa: NetLogo y Java

La siguiente tabla se basa en el trabajo implementado y comprobado en el proyecto.

Característica	NetLogo	Java
Facilidad de implementación	Se desarrollaron módulos de lectura, validación y reproducción, junto con widgets y personas. No se programó aquí la dinámica SIR.	Se programaron las clases del modelo, transiciones, validaciones, exportadores y coordinación de la aplicación.
Manejo de agentes	Representa personas con ID y estado importados en zonas A/B; posiciones fijas, sin calcular contactos o contagios.	Cada Individuo conserva ID y estado; SimulacionSIR selecciona contactos y actualiza infección y recuperación.
Generación de datos	Importa y valida los CSV; reproduce las trayectorias existentes.	Genera CSV agregado, CSV individual, PNG y parámetros con semilla, tiempo y huellas SHA-256.
Visualización	Cinco gráficas, monitores S/I/R y una vista de personas A/B, con reproducción progresiva y completa.	Exporta la curva completa en PNG y aporta el formulario y los controles Swing del visor integrado.
Flexibilidad	Permite cargar escenarios, comparar A/B y controlar su reproducción sin recalcular estados.	La terminal modifica N, I_0, β, γ, pasos, contactos y semilla. El formulario utiliza un contacto al generar.
Tiempo de ejecución	No se midió un tiempo de cálculo SIR equivalente: realiza importación, validación y reproducción.	Registra modelo_ms para el conteo inicial y los pasos. Excluye inicialización, exportaciones, PNG e interfaz.
Integración con otras herramientas	Usa la extensión csv, módulos .nls e InterfaceComponent.	Utiliza Maven y carga por reflexión NetLogo instalado; el JAR no incluye su distribución.

La tabla compara funciones complementarias. No permite declarar que una plataforma sea más rápida, más fácil en cualquier contexto o mejor para todos los modelos. El desarrollo de este proyecto requirió tanto lógica Java como programación de importación y presentación en NetLogo

6.3. Tiempo registrado y visualización

Los archivos nuevos registraron las siguientes mediciones parciales de Java:

Ejecución	modelo_ms (milisegundos)
Escenario base	3.028220
Menor contagio	2.947703
Mayor recuperación	2.416261
Otra semilla	2.713198

El controlador suma el tiempo del conteo inicial y de las llamadas paso(). La creación inicial del modelo, la escritura individual, los demás archivos y NetLogo quedan fuera de esa medición. Son valores de una ejecución por escenario, sensibles al entorno y al estado de la JVM; no forman un estudio de rendimiento.

La pausa del visor, configurable entre 20 y 500 ms, regula la velocidad visible y tampoco mide el coste epidemiológico. Java ya terminó de calcular cuando NetLogo reproduce los datos.

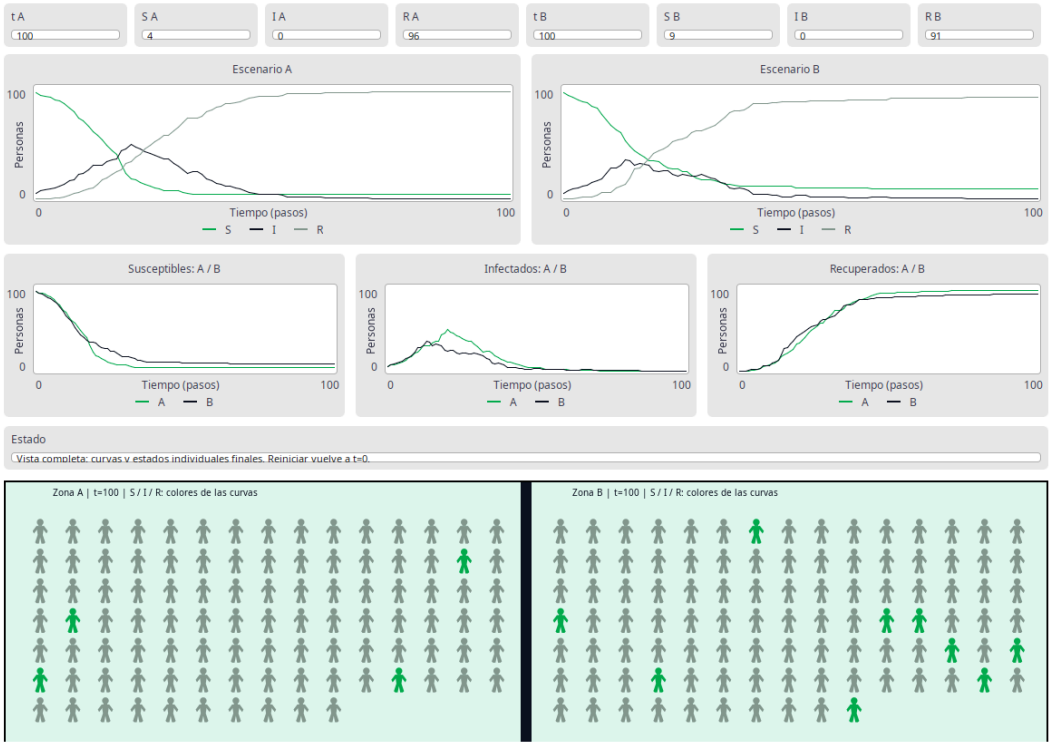


Figura 2. Captura nueva del componente NetLogo completo durante la prueba gráfica. A muestra la semilla 42 y B la semilla 7; en t=100 aparecen 96 y 91 recuperados respectivamente

La figura combina las cinco gráficas con una sola vista dividida en A/B. Las personas reproducen identidades y estados reales; verde indica susceptible, oscuro infectado y gris verdoso recuperado. La cuadrícula es una disposición visual y no una red de contactos.

6. Preguntas de Control

- **¿Qué representa el modelo SIR?**

  Representa la evolución de una población dividida en susceptibles, infectados y recuperados. Permite estudiar cómo las reglas de transmisión y recuperación modifican los grupos[1]. En esta práctica se concreta mediante individuos y probabilidades por paso, no mediante ajuste de datos clínicos.

- **¿Qué representan los estados S, I y R?**

  S reúne a quienes pueden infectarse; I, a quienes están infectados y pueden transmitir; R, a quienes se recuperaron y permanecen fuera de la transmisión[2]. En el programa cada persona conserva un ID y solo puede avanzar S→I→R.

- **¿Qué función cumplen los parámetros β y γ?**

  β controla la probabilidad de contagio cuando un infectado selecciona a un susceptible. γ controla la probabilidad de recuperación de un infectado original del paso. Ambos pertenecen al intervalo [0,1]; no se interpretan aquí como tasas continuas ni como porcentajes observados de pacientes.

- **¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?**

  Si se compararan implementaciones independientes, compartir población, condiciones iniciales, horizonte y reglas permitiría atribuir diferencias a aspectos concretos de implementación. También habría que controlar el orden de actualización[3]. Esta práctica utiliza dos herramientas con funciones diferentes y no ejecuta tres simuladores independientes.

- **¿Por qué Python, Java y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?**

  Pueden variar el generador pseudoaleatorio, el orden de llamadas, la activación de agentes y la interpretación temporal de las reglas. La misma semilla numérica no garantiza la misma secuencia entre lenguajes. Dentro de Java, la documentación de Random vincula la reproducción a la misma semilla y secuencia de operaciones (Oracle, s. f.). En este proyecto NetLogo reproduce los CSV Java, por lo que no debe producir otra trayectoria.

- **¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?**

  Dispone de agentes, razas, variables, formas, monitores y gráficas que facilitan expresar y observar estados individuales (NetLogo, s. f.-b). Aquí esas capacidades permiten representar personas por ejecución e ID. La presencia de figuras no significa que los contagios se calculen en NetLogo.

- **¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?**

  Organiza procesos, eventos y recursos compartidos mediante Python, lo que resulta útil cuando interesa programar sucesos y esperas en un reloj de simulación (SimPy, s. f.).

- **¿Qué ventajas ofrece Java para implementar modelos de simulación?**

  En esta aplicación permite organizar estados y responsabilidades en clases, parametrizar un generador reproducible, validar datos y exportar archivos sin dependencias de producción adicionales. También proporciona Swing para integrar controles con

NetLogo. Estas capacidades no prueban una superioridad universal de velocidad frente a otros lenguajes.

- **¿Qué función cumple el archivo CSV en la integración Python/Java→ NetLogo?**
  Actúa como contrato de intercambio entre el cálculo y la visualización. En esta aplicación Java produce los CSV y NetLogo los lee: el agregado alimenta curvas y monitores; el individual conserva IDs y estados. La documentación de la extensión contempla lectura por filas (NetLogo, s. f.-a). No se generaron CSV desde Python para la simulación.

- **¿Qué ventajas presenta separar la generación de datos de la visualización?**
  Permite revisar y conservar resultados, repetir su reproducción sin consumir nuevos sorteos, comparar ejecuciones y cambiar la presentación sin alterar las reglas epidemiológicas. Además, facilita verificar que personas, monitores y curvas coincidan en cada instante.

7. Conclusiones

Se ha podido concluir que:

- Java ha permitido organizar las reglas del modelo mediante clases y métodos, calcular los cambios de estado y exportar los resultados, calculando la evolución del modelo, mientras que NetLogo permitió representar visualmente los resultados mediante procedimientos, agentes, monitores y gráficas.
- Las ejecuciones nuevas conservaron 100 personas en cada paso y produjeron 101 conteos agregados y 10100 registros individuales por escenario. Sus series coincidieron con los ejemplos existentes; las 300 pruebas distintas terminaron sin fallos, errores u omisiones.
- La base alcanzó un máximo de 49 infectados en t=20 y terminó con 96 recuperados. Las variantes con menor β o mayor γ mostraron picos y totales de infección menores en las trayectorias observadas. Cambiar la semilla también modificó la evolución, lo que evidencia el carácter estocástico.

8. Bibliografía

[1] K. Bajracharya and R. Duboz, “Comparison of three agent-based platforms on the basis of a simple epidemiological model (WIP),” in Proc. Symp. Theory Model. Simul.—DEVS Integrative M&S Symp., San Diego, CA, USA, 2013, pp. 1–6. [Online]. Available: https://dl.acm.org/doi/10.5555/2499634.2499641

[2] T. Warnke, O. Reinhardt, and A. M. Uhrmacher, “Population-based CTMCs and agent-based models,” in Proc. 2016 Winter Simul. Conf. (WSC), 2016, pp. 1253–1264, doi: 10.1109/WSC.2016.7822181.

[3] W. M. Getz, R. Salter, O. Muellerklein, H. S. Yoon, and K. Tallam, “Modeling epidemics: A primer and Numerus Model Builder implementation,” Epidemics, vol. 25, pp. 9–19, Dec. 2018, doi: 10.1016/j.epidem.2018.06.001.

9. Anexos

Declaración de uso de IA

Herramienta utilizada: ChatGPT.

Forma de uso: La herramienta fue utilizada como apoyo conceptual para comprender el modelo SIR y su representación en NetLogo, incluyendo la aleatoriedad, la interpretación de curvas y la importación de CSV generados por Java. Esta consulta pidió explicaciones y ejemplos hipotéticos, sin generar ni modificar código. Las decisiones de implementación, la comprensión del programa, la comprobación de su funcionamiento y la interpretación de los resultados son responsabilidad del grupo.

Prompt:

Ayúdanos a comprender el funcionamiento del modelo SIR y su representación en NetLogo. Es nuestra primera experiencia con esta herramienta.

Ya leímos la guía de la práctica. Nosotros nos encargamos de programar y tomar las decisiones del proyecto; necesitamos explicaciones que nos permitan comprender las alternativas y justificar lo que implementemos.

NO generes código, archivos ni resuelvas la práctica completa. Queremos explicaciones paso a paso, ejemplos hipotéticos y preguntas que nos ayuden a razonar.

CONTEXTO

Trabajamos con Java 21 y NetLogo 7.0.4. Los parámetros iniciales son:

- Población: 100 individuos.
- Susceptibles: 95.
- Infectados: 5.
- Recuperados: 0.
- Beta: 0.30.
- Gamma: 0.10.
- Duración: 100 pasos.

Java calcula los contagios y las recuperaciones y exporta los resultados en CSV. NetLogo únicamente importa y representa esos datos: no genera nuevos contagios ni ejecuta una segunda simulación independiente.

EXPLÍCANOS

1. Qué representan S, I y R, cuáles son los supuestos del modelo y cómo interpretar beta y gamma. Distingue las tasas del modelo matemático de las probabilidades por contacto o por paso.

2. Cómo ocurre el contagio y la recuperación en una simulación discreta. Utiliza una población pequeña en un ejemplo hipotético y explica por qué importa separar el estado actual del siguiente.

3. Qué función cumplen la aleatoriedad y la semilla. Explica qué condiciones permiten reproducir una ejecución y por qué dos implementaciones independientes podrían producir resultados diferentes.

4. Cómo interpretar las curvas S(t), I(t) y R(t): qué significa el máximo de infectados, qué ocurre cuando ya no quedan infectados y qué conclusiones no podemos obtener de una única ejecución.

5. Cómo se organiza NetLogo y qué función cumplen observer, turtles, patches, ticks, botones, monitores y gráficas. Relaciona estos conceptos con la representación de nuestro SIR.

6. Cómo importar conceptualmente el CSV y reproducir sus filas progresivamente, manteniendo sincronizados el tiempo, los conteos y las curvas. Distingue el tiempo simulado de la velocidad de reproducción y explica el funcionamiento esperado de cargar, pausar, continuar y reiniciar.

7. Cómo representar los estados mediante agentes de colores. Distingue entre visualizar cantidades agregadas y reconstruir la trayectoria real de cada individuo: explica qué información adicional necesitaríamos si el CSV solo contiene tiempo y conteos S/I/R.

8. Cómo comprobar la conservación de la población, los límites de los parámetros y la coincidencia entre el CSV y NetLogo. Explica por qué visualizar el mismo archivo en dos herramientas no equivale a comparar dos simulaciones independientes y por qué cambiar beta o gamma requiere generar nuevos datos en Java.

FORMA DE RESPONDER

Usa palabras claras y ejemplos identificados como hipotéticos, sin presentarlos como resultados de nuestro proyecto. Diferencia la verificación del programa de la validación del modelo.

Si adjuntamos archivos, basa las explicaciones en su contenido real y ayúdanos a localizar los conceptos en el código, sin modificarlo. No inventes funcionalidades ni supongas que están implementadas.

Para instrucciones específicas de NetLogo 7.0.4, utiliza documentación oficial y señala lo que no puedas comprobar.

Link del video:

APE04-1C-Simulacion-5toCiclo

Link del repositorio:
https://github.com/AndersonC15/Simulacion/tree/main/04APE_Grupal